```
import networkx as nx

# Create a graph
G = nx.Graph()
G.add_edges_from([(1, 2), (1, 3), (2, 4), (2, 5), (3, 6)])

# Perform DFS and get edges
dfs_edges = list(nx.dfs_edges(G, source=1))
print(f"DFS Edges: {dfs_edges}")

# Get the DFS tree
dfs_tree = nx.dfs_tree(G, source=1)
print(f"DFS Tree nodes: {list(dfs_tree.nodes())}")
```

```
DFS Edges: [(1, 2), (2, 4), (2, 5), (1, 3), (3, 6)]
DFS Tree nodes: [1, 2, 4, 5, 3, 6]
```

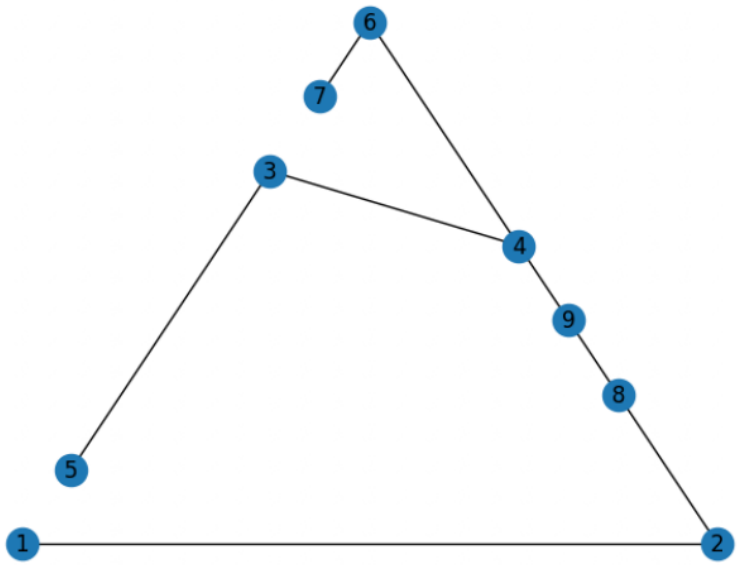
```
import networkx as nx

# Create a graph
G = nx.Graph()
G.add_edges_from([(1, 2), (1, 3), (2, 4), (2, 5), (3, 6)])

print(dict(G.degree)[2])
```

```
3
```

```
import networkx as nx

# Create a graph
G = nx.DiGraph()
G.add_edges_from([(1, 2), (1, 3), (2, 4), (2, 5), (3, 6)])

print(dict(G.out_degree)[2])
print(dict(G.in_degree)[2])
```

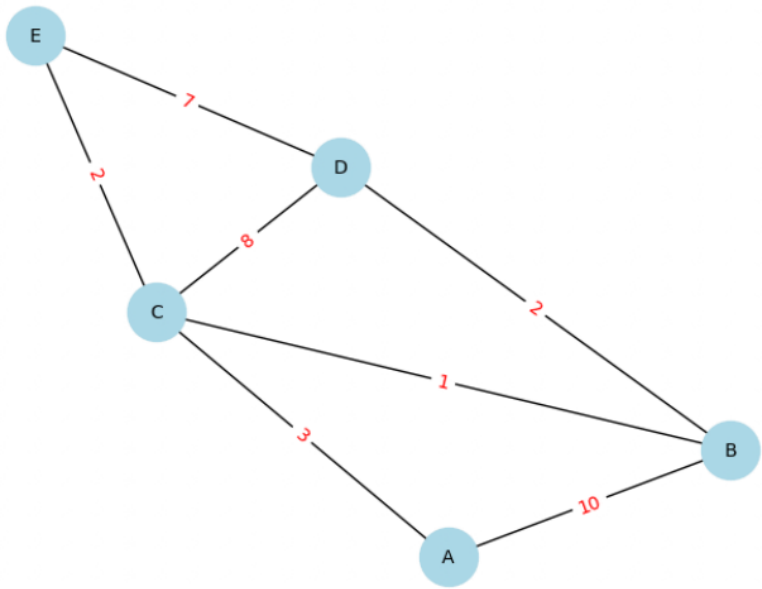
```
2
1
```

```
import networkx as nx

#edge_list = [(1,2),(2,3),(3,4),(3,5),(4,6),(6,7),(2,8),(8,9),(9,4)]
edge_list = [(1,2),(3,4),(3,5),(4,6),(6,7),(2,8),(8,9),(9,4)]

G = nx.Graph()
G.add_edges_from(edge_list)

nx.draw_planar(G, with_labels=True)
plt.show()

print(nx.shortest_path(G,2,4))
```




```
[2, 8, 9, 4]
```

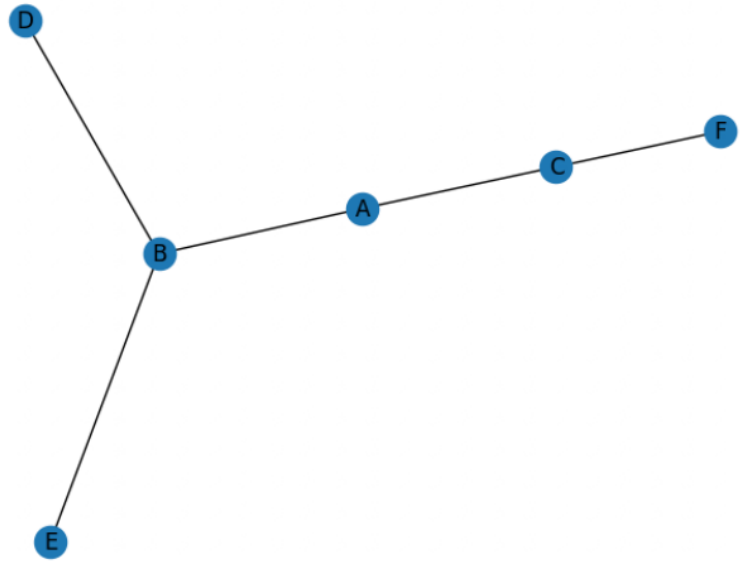
```
import networkx as nx

G = nx.Graph()
G.add_edge('A', 'B', weight=10)
G.add_edge('A', 'C', weight=3)
G.add_edge('B', 'C', weight=1)
G.add_edge('B', 'D', weight=2)
G.add_edge('C', 'D', weight=8)
G.add_edge('C', 'E', weight=2)
G.add_edge('D', 'E', weight=7)

#nx.draw_planar(G, with_labels=True)
#plt.show()

pos = nx.spring_layout(G)
edge_labels = nx.get_edge_attributes(G, 'weight')
nx.draw(G, pos, with_labels=True, node_color='lightblue', node_size=1000, font_size=10)
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_color='red')
plt.show()

path = nx.shortest_path(G, source='A', target='E', weight='weight')
print(f"Shortest path from A to E: {path}")

length = nx.shortest_path_length(G, source='A', target='E', weight='weight')
print(f"Shortest path length from A to E: {length}")

distances, paths = nx.single_source_dijkstra(G, source='A', weight='weight')
print(f"Distances from A: {distances}")
print(f"Paths from A: {paths}")
```




```
Shortest path from A to E: ['A', 'C', 'E']
Shortest path length from A to E: 5
Distances from A: {'A': 0, 'C': 3, 'B': 4, 'E': 5, 'D': 6}
Paths from A: {'A': ['A'], 'B': ['A', 'C', 'B'], 'C': ['A', 'C'], 'D': ['A', 'C', 'B', 'D'], 'E': ['A', 'C', 'E']}
```

```
import networkx as nx

# Create a sample adjacency list file
with open("graph.adjlist", "w") as f:
    f.write("A B C\n")
    f.write("B D E\n")
    f.write("C F\n")
    f.write("D\n") # Node D has no outgoing edges in this example

# Read the adjacency list from the file to create a graph
# create_using can be nx.Graph() for undirected or nx.DiGraph() for directed
G = nx.read_adjlist("graph.adjlist", create_using=nx.Graph())

print("Nodes:", G.nodes())
print("Edges:", G.edges())

nx.draw_spring(G, with_labels=True)
plt.show()
```

```
Nodes: ['A', 'B', 'C', 'D', 'E', 'F']
Edges: [('A', 'B'), ('A', 'C'), ('B', 'D'), ('B', 'E'), ('C', 'F')]
```




```
import networkx as nx

# Create a graph
G = nx.Graph()
G.add_edges_from([(1, 2), (1, 3), (2, 4), (2, 5), (3, 6)])

# Perform DFS and get edges
dfs_edges = list(nx.dfs_edges(G, source=1))
print(f"DFS Edges: {dfs_edges}")

# Get the DFS tree
dfs_tree = nx.dfs_tree(G, source=1)
print(f"DFS Tree nodes: {list(dfs_tree.nodes())}")
```

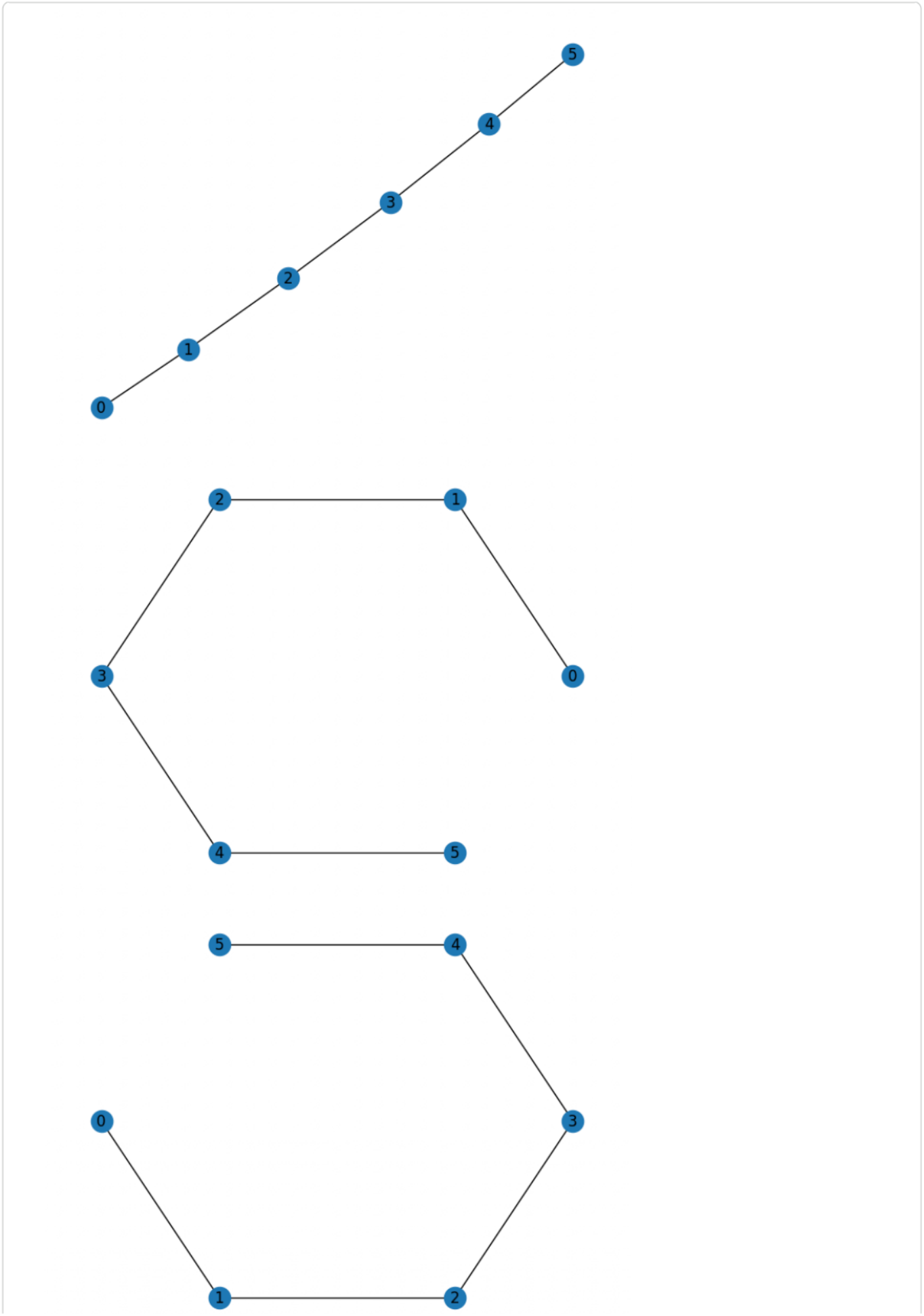
```
import networkx as nx

# Create a graph
G = nx.Graph()
G.add_edges_from([(1, 2), (1, 3), (2, 4), (2, 5), (3, 6)])

# Perform BFS and get edges
bfs_edges = list(nx.bfs_edges(G, source=1))
print(f"BFS Edges: {bfs_edges}")

# Get the BFS tree
bfs_tree = nx.bfs_tree(G, source=1)
print(f"BFS Tree nodes: {list(bfs_tree.nodes())}")
```

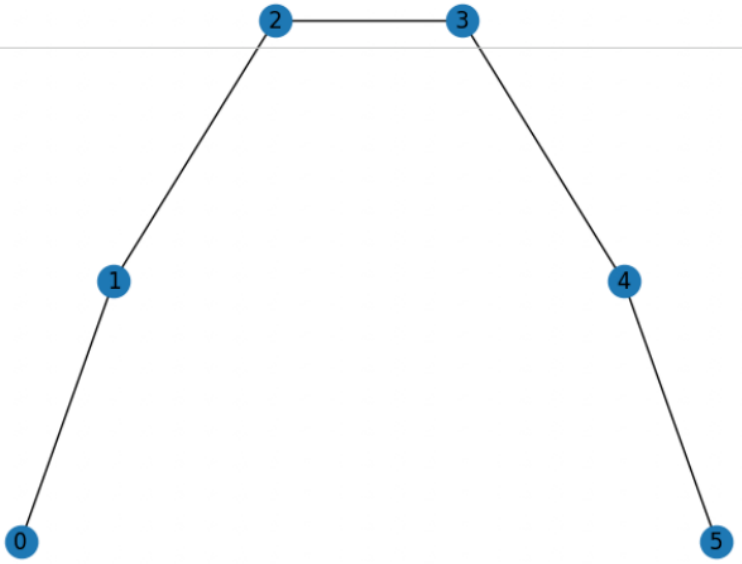
```
BFS Edges: [(1, 2), (1, 3), (2, 4), (2, 5), (3, 6)]
BFS Tree nodes: [1, 2, 3, 4, 5, 6]
```

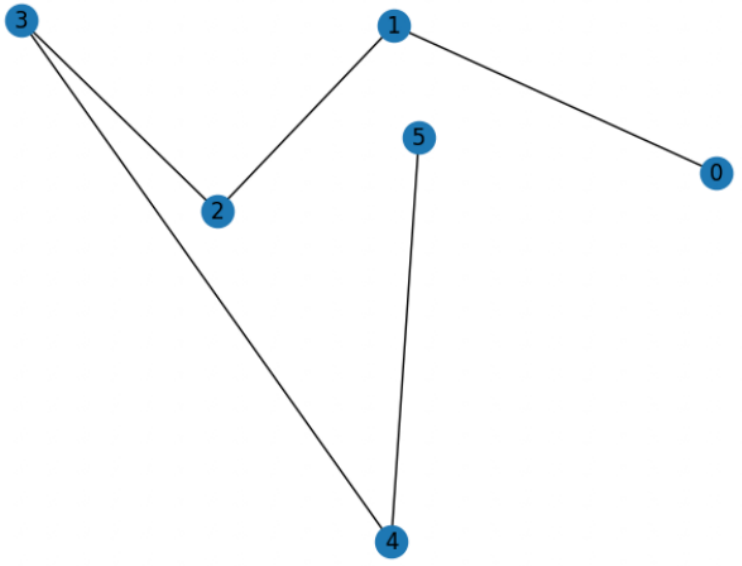
```
import networkx as nx
import matplotlib.pyplot as plt

# Create a sample graph
G = nx.path_graph(6)  # A simple line graph: 0--1--2--3--4--5


# Standard DFS from source node 0
print("Full DFS edges:", list(nx.dfs_edges(G, source=0)))

# DFS with a depth limit of 2 from source node 0
print("Limited DFS edges:", list(nx.dfs_edges(G, source=0, depth_limit=2)))
```

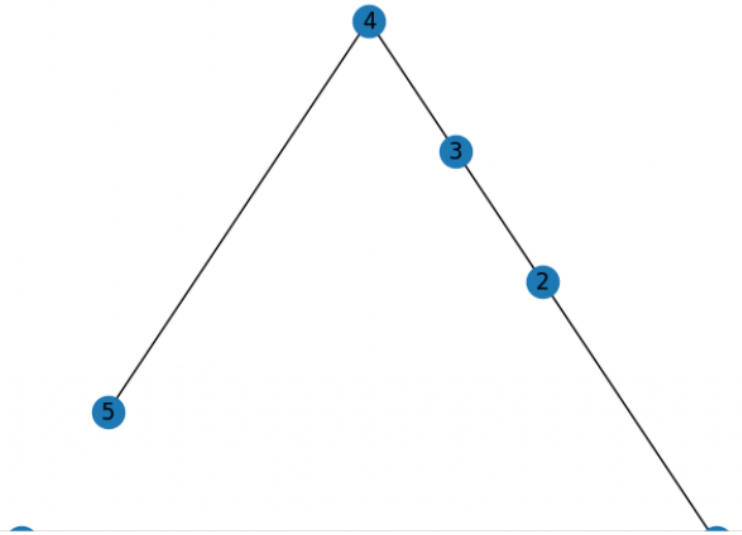
```
Full DFS edges: [(0, 1), (1, 2), (2, 3), (3, 4), (4, 5)]
Limited DFS edges: [(0, 1), (1, 2)]
```

```
import networkx as nx
import matplotlib.pyplot as plt

# Create a sample graph
G = nx.path_graph(6)  # A simple line graph: 0--1--2--3--4--5
nx.draw_spring(G, with_labels=True)
plt.show()

nx.draw_circular(G, with_labels=True)
plt.show()

nx.draw_shell(G, with_labels=True)
plt.show()

nx.draw_spectral(G, with_labels=True)
plt.show()

nx.draw_random(G, with_labels=True)
plt.show()

nx.draw_planar(G, with_labels=True)
plt.show()



# Standard DFS from source node 0
print("Full DFS edges:", list(nx.dfs_edges(G, source=0)))

# DFS with a depth limit of 2 from source node 0
print("Limited DFS edges:", list(nx.dfs_edges(G, source=0, depth_limit=2)))
```

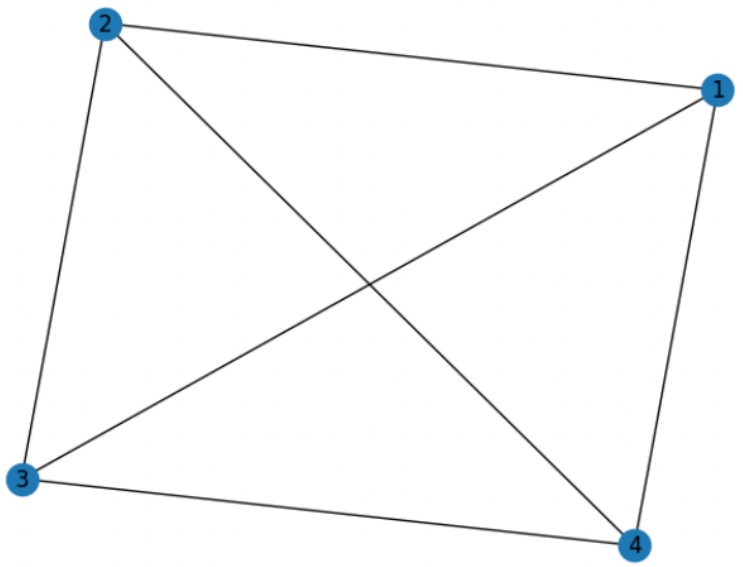

5
4
3
2
1
0
2
1
3
0
4
5
5
4
0
3
1
2

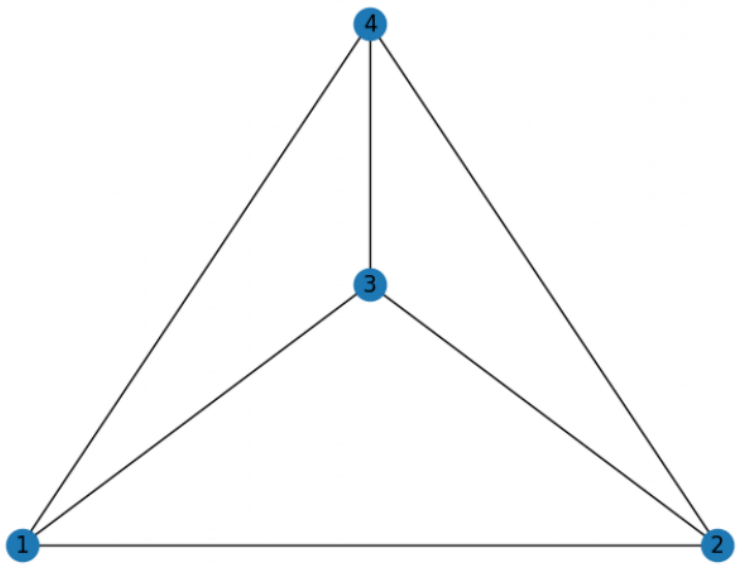

2
3
1
4
0
5


3
1
5
0
2
4


4
3
2
5

```
import networkx as nx
import matplotlib.pyplot as plt

# Create a sample graph
G = nx.Graph()
G.add_edges_from([(1,2),(1,3),(1,4),(2,3),(2,4),(3,4)])

nx.draw_spring(G, with_labels=True)
plt.show()

nx.draw_planar(G, with_labels=True)
plt.show()
```






```
import networkx as nx
import matplotlib.pyplot as plt

# Create a sample graph
G = nx.complete_graph(5)
#G.add_edges_from([(1,2),(1,3),(1,4),(1,5),(2,3),(2,4),(2,5),(3,4),(3,5),(4,5)])
```